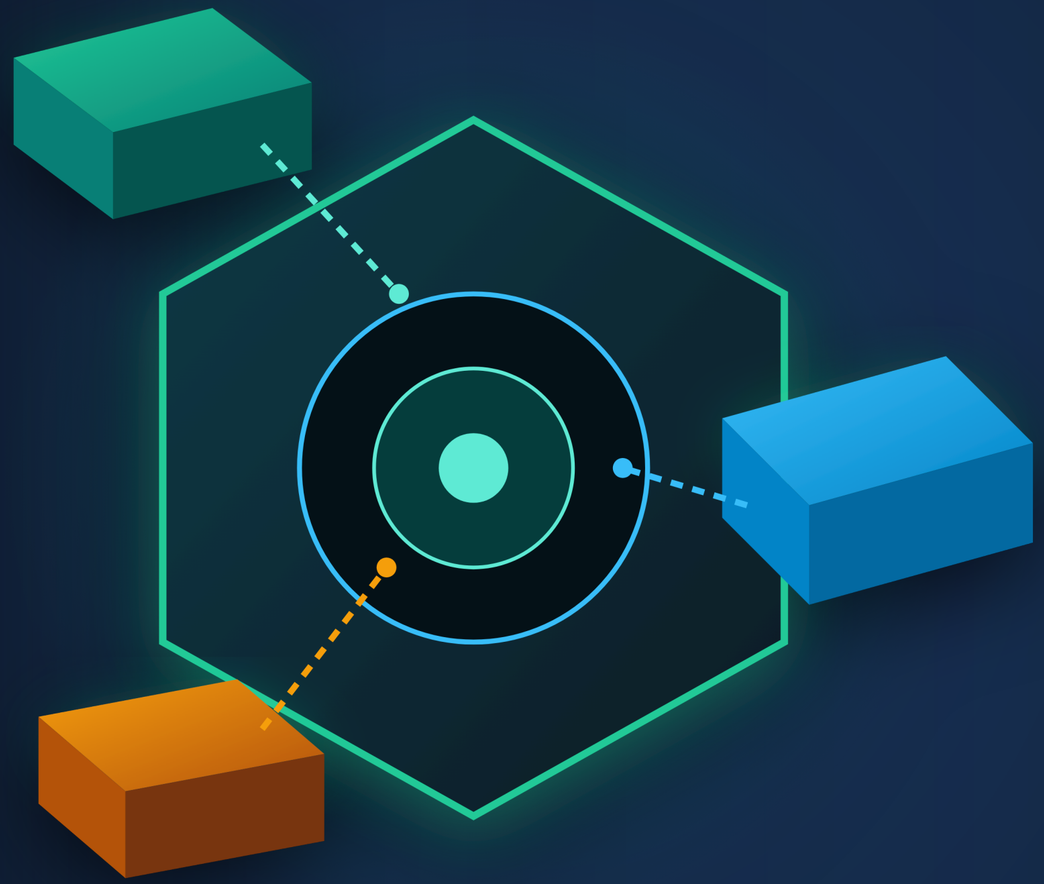
Software Engineering Lab

Lab 16

Dependency Injection

Injeksi Dependensi dengan Go Interfaces & Provider



Lukman / lukman-ss

Kegagalan Komponen Tanpa DI

Hard-coded dependencies menyebabkan test flakiness & scaling issues.

MASALAH NYATA

Komponen menginisialisasi dependensi internal secara langsung.

Mocking menjadi mustahil, sehingga unit test tidak isolasi.
Perubahan implementasi memaksa rebuild seluruh service.

Refactor Manual Cepat Menyisakan Leak

Developer mengganti konstruktor tapi melupakan wiring di provider.

KEGAGALAN

Instansi tetap dibuat di tempat lama.

Provider tidak di-update, menyebabkan nil panic.
Tidak ada compile-time guard untuk missing bindings.

Graf DI sebagai DAG

Nodes = Interfaces, Edges = Providers, acyclic guarantee.

DI GRAPH

Alur Injeksi

Parser → Provider Registry → Resolver → Consumer

Contoh Provider

Registrasi sederhana dengan func.



CODE PATH

Path Failure: Missing Binding

Resolver panic when binding absent.

ALUR

Consumer requests InterfaceX.

Resolver looks up binding → not found.
panic: no provider for InterfaceX.

DI vs Tanpa DI

Efek pada testability & startup time.

3 Keputusan Kunci

When to bind, scope, and replace.

Bind per request vs singleton.

Prefer interface over concrete.
Use lazy provider for heavy deps.

Demo Terminal

Running demo shows injected service prints OK.

Yang Tidak Dibuktikan

DI tidak mengatasi network latency.

Runtime errors beyond binding not covered

Performance impact not measured

Keuntungan vs Overhead

Cleaner code vs extra indirection.

Improved testability

Slight latency

More boilerplate

Goroutine Safe Registry

Mutex protects binding map.



CODE PATH

Sebelum & Sesudah

Hard-coded vs injected.

Test Coverage

Key assertion from processor_test.go

Demonstrates mock replacement

Shows panic recovery

Mock Injection

Key assertion from processor_test.go

Demonstrates mock replacement

Shows panic recovery

Error Propagation

Key assertion from processor_test.go

Demonstrates mock replacement

Shows panic recovery

Performance Benchmark

Key assertion from processor_test.go

Demonstrates mock replacement

Shows panic recovery

Reflection Safety

Key assertion from processor_test.go

Demonstrates mock replacement

Shows panic recovery

Implementasi DI

Langkah-langkah siap.

- ✓ Define interfaces
- ✓ Create providers
- ✓ Register in Registry
- ✓ Resolve in consumers

Service Locator Abuse

Global locator hides dependencies.

Hard to track actual wiring

Testing suffers

Prefer constructor injection

Perintah Utama

go run cmd/demo/main.go && go test ./...

Inversi Dependensi Sepanjang Siklus Hidup

Keep abstractions stable, implementations fluid.

Terima Kasih

Software Engineering Lab 16 - Dependency Injection

Commit: $(git rev-parse --short HEAD)

© Lukman

REPOSITORY

`github.com/lukman-ss/software-engineering-lab/tree/main/labs/16-dependency-injection`

Lukman / lukman-ss

Commit: $(git rev-parse --short HEAD)

© Lukman